

SOLUTION EVALUATION:

Il lavoro non finisce con la creazione di una soluzione funzionante. Prima di concludere, è fondamentale valutare la qualità della soluzione. Ciò implica rispondere a quattro domande principali:

1. **È corretta?** Risolve effettivamente il problema?
2. **È efficiente?** Utilizza le risorse in modo ragionevole?
3. **È elegante?** È semplice ed efficace?
4. **È utilizzabile?** Offre un'esperienza soddisfacente per gli utenti?

IS IT CORRECT?

La domanda cruciale è se la soluzione risolve effettivamente il problema originale. Se non lo fa, tutte le altre misure di qualità sono irrilevanti. Un approccio prudente è assumere che un programma sia errato finché non si dimostra il contrario, poiché la maggior parte delle soluzioni iniziali necessita di test e debug. La correttezza di un programma può essere provata matematicamente o empiricamente, attraverso test che verificano che la soluzione soddisfi i requisiti specifici. Un singolo test fallito dimostra che la soluzione è errata. I test empirici non provano la correttezza in modo definitivo, ma offrono prove che vanno da nessuna prova a una corretta soluzione per tutti i possibili input. In definitiva, se anche un solo test fallisce, la soluzione è considerata errata, indipendentemente dalla sua efficienza o complessità.

IS IT EFFICIENT?

Ogni algoritmo richiede risorse per eseguire il suo compito, e la sua efficienza viene misurata principalmente in termini di tempo e spazio:

- **Tempo:** la durata dell'esecuzione dell'algoritmo, misurata dal numero di passaggi effettuati.
- **Spazio:** la quantità di memoria necessaria per eseguire l'algoritmo.

L'efficienza di un algoritmo dipende quindi dal suo utilizzo di risorse, e se tale utilizzo è accettabile, l'algoritmo può essere considerato efficiente. Tuttavia, confrontare in modo affidabile l'efficienza di diversi algoritmi può essere difficile, poiché gli algoritmi reagiscono diversamente in base ai dati di input, come ad esempio la dimensione dell'elenco su cui operano, che influisce sul tempo di esecuzione. L'efficienza di un algoritmo può essere misurata analizzando il suo comportamento in base alla dimensione dei dati di input. Gli algoritmi vengono classificati in **classi di complessità**, che descrivono l'uso delle risorse nel caso peggiore. L'efficienza viene espressa tramite la notazione **Big-O** (O grande), che indica quanto tempo o memoria un algoritmo impiega in relazione alla dimensione dell'input. Ad esempio, **O(N)** indica che il tempo di esecuzione cresce linearmente con l'input, mentre **O(N²)** indica una crescita quadratica. Queste categorie permettono di confrontare l'efficienza degli algoritmi e prevedere come si comportano con input più grandi.

Example: Linear search

Un algoritmo di ricerca lineare cerca un elemento in un elenco confrontando ciascun elemento con l'elemento di ricerca. La sua complessità dipende dai seguenti casi:

- **Miglior caso ($O(1)$):** l'elemento di ricerca è il primo dell'elenco.
- **Caso medio ($O(N/2)$):** l'elemento di ricerca è, in media, nel mezzo dell'elenco.
- **Caso peggiore ($O(N)$):** l'elemento di ricerca è l'ultimo o non viene trovato.

Gli algoritmi con prestazioni nel caso peggiore come $O(N)$ o $O(\log N)$ sono considerati efficienti, mentre quelli con prestazioni come $O(2N)$ o $O(N!)$ sono inefficienti, poiché i loro tempi di esecuzione aumentano rapidamente con l'aumentare della dimensione dell'input.

IS IT ELEGANT?

L'eleganza in una soluzione software, pur essendo un concetto meno oggettivo rispetto a correttezza ed efficienza, riguarda l'abilità di risolvere un problema in modo semplice ed efficace. Un esempio di eleganza in ingegneria è la trappola per prevenire cattivi odori, che risolve il problema con una soluzione semplice e senza bisogno di manutenzione. In matematica, l'eleganza si manifesta quando una soluzione complessa è ridotta a una formula semplice, come nel caso della somma degli interi di Gauss, che trova una soluzione rapida e semplice attraverso un'osservazione intelligente.

L'eleganza in informatica implica trovare soluzioni che richiedano meno risorse e tempo. Tuttavia, soluzioni non ovvie possono essere difficili da comprendere per altri e richiedono spiegazioni chiare per essere facilmente manutenibili. È importante non cercare di essere ingegnosi solo per il gusto di esserlo, ma piuttosto giustificare sempre il motivo per cui una soluzione più complessa è stata adottata.

IS IT USABLE?

L'usabilità di una soluzione software misura quanto sia facile e soddisfacente per gli utenti utilizzarla per raggiungere i propri obiettivi. A differenza di criteri come correttezza ed efficienza, che valutano la soluzione in termini funzionali, l'usabilità si concentra sulle esigenze umane, come la facilità di apprendimento, l'efficienza, la memoria, gli errori e la soddisfazione degli utenti. Gli utenti desiderano soluzioni che siano facili da imparare, intuitive, indulgenti verso gli errori e richiedano poco sforzo.

Le componenti misurabili dell'usabilità includono:

1. **Apprendibilità:** quanto è facile iniziare a usare la soluzione.
2. **Efficienza:** quanto velocemente gli utenti completano i compiti dopo aver imparato.
3. **Memorabilità:** quanto facilmente gli utenti ritrovano competenza dopo un periodo di inattività.
4. **Errori:** quante difficoltà incontrano gli utenti, la gravità degli errori e quanto facilmente possono recuperare.
5. **Soddisfazione:** quanto piacevole è l'esperienza d'uso.

Per valutare l'usabilità, si effettuano test osservazionali con utenti rappresentativi del pubblico target, che completano compiti specifici mentre il valutatore osserva e annota successi e difficoltà. Al termine del test, viene somministrato un questionario che misura vari aspetti dell'esperienza utente, come la facilità d'uso e la soddisfazione generale.

Il test di usabilità può anche essere fatto con prototipi iniziali, il che permette di identificare e risolvere i problemi più rapidamente.

TRADE-OFFS

Le soluzioni software spesso comportano compromessi, come quello spazio-tempo, dove migliorare un aspetto può peggiorare un altro. Ad esempio, nella compressione delle immagini, ridurre lo spazio richiesto per memorizzare i dati (es. usando la compressione) può aumentare il tempo necessario per elaborare i dati (poiché richiede più passaggi). Allo stesso modo, altre soluzioni potrebbero bilanciare ingegnoseria e comprensibilità o usabilità e flessibilità. Questi compromessi dipendono dalle priorità del progetto: ad esempio, se si inviano dati su una rete, può essere utile ottimizzare lo spazio a scapito di un maggiore tempo di elaborazione. È importante definire in anticipo quali aspetti ottimizzare per prendere decisioni consapevoli sui compromessi da fare.